

Backlog del Sprint 2 — Scaps

Catálogo, ficha de producto y visor 3D

Sprint 2 · 13 al 26 de julio de 2026. Con la API de productos y la sesión funcionando desde el Sprint 1, este sprint construye lo que ve cualquier visitante: el visor 3D del producto destacado en la landing, el catálogo con búsqueda, filtros y orden, y la ficha de producto con su galería y la opción **Ver en 3D**, más lo que enmarca todas las pantallas: el navbar responsive, con logo, y el footer con los créditos de los modelos 3D. Es un sprint de frontend: todo lo que consume ya está desplegado en Render, así que no hay endpoints nuevos ni migraciones. Al terminar tiene que estar desplegado en Vercel: cualquiera puede entrar, rotar el destacado, recorrer el catálogo y abrir la ficha de cualquier producto.

Cada tarea indica su área entre corchetes, sus labels, qué hacer, el criterio para darla por terminada y, cuando aplica, de qué depende. La forma de trabajo (columnas del tablero, flujo y reglas) está en `Sprint-0-Scaps.pdf` y no se repite acá.

Antes de arrancar: la guía visual es una referencia, no la especificación.

`Guía_visual_Scaps_e-commerce.pdf` marca la estética —colores, tipografía, espaciados y la disposición general de cada pantalla—, pero lo que se construye lo definen estas tareas. En algunos puntos se apartan de la guía a propósito, y **si una tarea y la guía no coinciden, vale la tarea**. Las diferencias, para que nadie las construya copiando la guía:

- **Ficha de producto:** no lleva SKU, “Envío 48h” ni el bloque de características (material, talla, color). El modelo de datos no tiene esos campos, el envío salió del MVP junto con el checkout, y color y talla son variantes, que el MVP no tiene. Si hace falta contar de qué está hecha una gorra, va en la descripción, que es texto libre. El breadcrumb tampoco tiene nivel de categoría (“Truckers”), porque no hay categorías.
- **Catálogo:** la card de un producto sin stock mantiene **Ver detalles** habilitado. Sin stock la ficha se puede mirar igual; lo que se deshabilita es Agregar al carrito, en el Sprint 3.
- **Imágenes de producto:** el área de cada imagen es **blanca**, no `#242424`, porque las fotos tienen fondo blanco; y en las cards del catálogo es **cuadrada**, no 4:5, porque las fotos también lo son. Se explica en la primera tarea.
- **Visor 3D:** solo rota. No hay zoom con la rueda del mouse ni botones + −, y el texto de ayuda es “Arrastrá para rotar”. Se explica en la tarea del componente.
- **Precios:** con el formato argentino, “\$ 15.999”, no “10.000\$”.

Y una regla para todas las pantallas: responsive al 100 %. Cada pantalla de este sprint se prueba con el modo dispositivo de las DevTools en cuatro anchos —360, 390, 768 y 1440 px— y en ninguno puede aparecer scroll horizontal ni un elemento cortado. Cada tarea dice qué cambia en el celular, y eso es parte del “hecho cuando”, no un pulido para después. Una consecuencia práctica: **nada de anchos fijos en píxeles**. Las medidas de la guía, como los 700 × 520 de la ficha, son tamaños máximos en escritorio; en pantallas chicas se achican manteniendo la proporción.

To Do — se pueden tomar ya (no dependen de nada)

[setup] Piezas compartidas de producto: tipos, precio e imagen

setup

prioridad-alta

- **Qué hacer:** crear tres piezas que van a usar la landing, el catálogo y la ficha, para que cada pantalla no arme su propia versión (el mismo motivo por el que existe el cliente HTTP del Sprint 1):
 - **Los tipos de TypeScript** de lo que devuelve la API de productos, copiados de las formas del contrato: `ProductCard`, el producto en formato card (lo que trae `GET /products`); `ProductDetail`, el producto en detalle (`GET /products/:slug` y `GET /products/featured`); `GalleryImage`, una imagen de la galería; y `Paginated<T>`, el sobre paginado `{ data, meta }`. La imagen no se llama `ProductImage` para no chocar con el componente. Los campos conservan los nombres del contrato, en español (`precio`, `es_principal`), y los montos se tipan como `string`, porque así viajan.
 - **Una función `formatPrice`** que pase el string de la API al formato argentino: `"15999.00"` → \$ 15.999 y `"15999.50"` → \$ 15.999,50. La resuelve `Intl.NumberFormat('es-AR', { style: 'currency', currency: 'ARS' })`, que ya viene en el navegador, mostrando los centavos solo cuando no son cero. Y una regla que arranca acá: **el frontend nunca hace cuentas con plata**. Convertir el string a número para mostrarlo está bien; los subtotales y el total del carrito los calcula el backend.
 - **Un componente `<ProductImage>`** para toda imagen de producto de la app: dibuja un área blanca con el radio de la guía (8 px) y adentro muestra la imagen **entera** (`object-fit: contain`), con el `alt` que viene de la API y `loading="lazy"` (el navegador la descarga recién cuando se acerca a la pantalla). Recibe la proporción del área como prop, porque el catálogo usa un área cuadrada (1:1) y la ficha una de 700 × 520. El área nunca tiene medidas fijas: la proporción se define con `aspect-ratio` y el ancho lo pone el contenedor, así se adapta a cualquier pantalla.
- **Por qué el área es blanca:** las fotos de producto son cuadradas y tienen fondo blanco, como las definió el Sprint 1. En el catálogo el área es cuadrada como ellas, así que la foto la llena justo. En la ficha el área es horizontal: la foto se muestra entera y el espacio que sobra a los costados, del mismo blanco, se confunde con su fondo, así que no se ve ningún borde y no se recorta nada de la gorra. Sobre la interfaz oscura, esas áreas blancas funcionan como una vitrina.
- **Hecho cuando:** las tres piezas están en `main`; `formatPrice` muestra \$ 15.999 para `"15999.00"` y \$ 15.999,50 para `"15999.50"`; y una imagen del seed mostrada con `<ProductImage>` en el área cuadrada del catálogo y en la de 700 × 520 de la ficha se ve entera, sin recortes y **sin ningún borde visible entre la foto y el área**.

Ojo con el blanco: tiene que ser exactamente el de las capturas. Si al renderizarlas el fondo quedó apenas corrido del `#FFFFFF`, se ve el borde de la foto. Abrí una de las imágenes de R2, medí el color del fondo con un cuentagotas (el de las DevTools o el de cualquier editor de imágenes) y usé ese valor. **Por qué va primero:** la grilla del catálogo, la ficha y la landing dependen de estas piezas. Es una tarea corta: conviene tomarla y mergearla en el día, porque hasta que esté, esas tres esperan.

[visor-3d] Componente del visor 3D

visor-3d

prioridad-alta

- **Qué hacer:** convertir el POC del Sprint 0 en un componente reutilizable, `<ModelViewer>`, que reciba la URL de un `.glb` y lo muestre para rotarlo con el mouse o con el dedo. Es un componente, no una pantalla: cada lugar que lo usa muestra su propia copia, con su propio modelo. Lo usan dos tareas de este sprint: la landing, donde el destacado se ve en 3D de entrada, y la ficha, donde reemplaza a la imagen grande solo al tocar Ver en 3D, sin salir de la página.

- **Tiene que resolver:**

- **El decodificador de Draco, servido por nosotros.** `useGLTF` de drei ya abre modelos comprimidos con Draco sin configurar nada, pero para eso baja el decodificador de un CDN de Google: una dependencia externa que no controlamos y que ya falló alguna vez. Copiar el contenido de `node_modules/three/examples/jsm/libs/draco/gltf/` a `apps/web/public/draco/` (con workspaces, `three` suele estar en el `node_modules` de la raíz) y apuntar drei ahí una sola vez con `useGLTF.setDecoderPath('/draco/')`. Son archivos del mismo `three` que usa el proyecto, así que la versión siempre coincide; si algún día se actualiza `three`, se vuelven a copiar.
 - **Iluminación declarada en el código**, por ejemplo una luz ambiente y una direccional. No usar los presets de `<Environment>` (`city`, `studio` ...) ni `<Stage>` con su configuración por defecto, que usa uno de esos presets: bajan su imagen de iluminación de un CDN externo, y si ese CDN falla, el visor queda roto.
 - **Encuadre automático.** Los cuatro modelos son de autores distintos y vienen con tamaños y centros distintos. Envolver el modelo en `<Center>` y `<Bounds fit clip observe>` de drei, que lo centran y acomodan la cámara según su tamaño: ninguno queda gigante, diminuto o corrido. Gracias a `observe`, el encuadre se recalcula cuando cambia el tamaño, por ejemplo al girar el celular. `<Bounds>` necesita que los controles tengan la prop `makeDefault`.
 - **Solo rotar:** `<OrbitControls makeDefault enableZoom={false} enablePan={false} />`. Sin zoom, la rueda del mouse sobre el visor sigue scrolleando la página en vez de acercar el modelo.
 - **Fondo transparente.** El canvas de R3F es transparente por defecto, así que el color lo pone el contenedor: blanco en la ficha, oscuro en la landing. No definir un color de fondo dentro de la escena.
 - **Carga y errores.** Mientras el `.glb` baja, un indicador visible (drei trae `<Html>` para poner texto dentro del canvas y `useProgress` para el porcentaje). Si el `.glb` no carga, `useGLTF` lanza un error, y en React un error que nadie atrapa desmonta la app entera: la pantalla queda en blanco. Por eso el `<Canvas>` va dentro de un *error boundary*, un componente que atrapa los errores de sus hijos y muestra otra cosa en su lugar. Se escribe como componente de clase (unas veinte líneas; el ejemplo está en la documentación de React), sin instalar ninguna librería. El visor avisa del error con una prop `onError`, para que cada pantalla decida qué mostrar.
 - **Carga diferida del código.** Importar el visor con `React.lazy(() => import('./ModelViewer'))` (el componente tiene que ser el export por defecto de su archivo). `three.js` pesa bastante, y así solo lo descargan las pantallas que muestran el visor: el catálogo no lo baja.
- **Hecho cuando:** los cuatro modelos cargan desde R2 en local y en el preview de un Pull Request de Vercel; en la pestaña *Network* de las DevTools el decodificador sale de nuestro dominio (`/draco/...`) y no hay ningún pedido a `gstatic.com` ni a otro CDN; los cuatro se ven centrados y de un tamaño parecido; arrastrar rota el modelo y la rueda del mouse scrollea la página; al cambiar el tamaño de la ventana el modelo se vuelve a encuadrar solo; y con una URL rota el visor dispara `onError` y el resto de la página sigue funcionando.

El error más común con R3F: el `<Canvas>` ocupa el 100 % de su contenedor, así que si el contenedor no tiene una altura definida, el canvas mide cero y no se ve nada, sin ningún error en la consola. Siempre darle alto al contenedor.

[setup] Logo, favicon, navbar responsive y rutas del MVP

setup prioridad-alta

- **Qué hacer:** dejar listo el marco que aparece en todas las pantallas. Son tres partes, en este orden: conseguir el logo y el favicon, limpiar las rutas y rehacer el navbar para que sea responsive.

- **1 • Logo y favicon.** Hoy la app no tiene ninguno de los dos, y hacen falta los dos, genéricos, de demo. Son archivos distintos aunque salgan del mismo diseño: el logo se ve en el navbar, y el favicon en la pestaña del navegador, a 16 o 32 px, donde un logo completo no se lee. Cómo se consiguen queda a criterio de quien tome la tarea; lo que está definido es el formato, el tamaño y dónde vive cada archivo.
- **Formatos y tamaños:**
 - **Logo del navbar:** SVG con fondo transparente, en un lienzo horizontal de entre 3:1 y 5:1 (por ejemplo, 140 × 28). Se muestra con alto fijo y ancho automático, así nunca se deforma: 24 px de alto en el celular y 28 px desde `md ( h-6 md:h-7 w-auto )`. A ese alto no debería pasar de unos 140 px de ancho, para que en el celular quede lugar para el botón del menú.
 - **Favicon:** cuadrado, en tres archivos. `favicon.svg`, que usan los navegadores modernos y se ve nítido a cualquier tamaño; `favicon-32x32.png`, de 32 × 32 px, para los navegadores que no aceptan un SVG como ícono; y `apple-touch-icon.png`, de 180 × 180 px, el ícono que usa el iPhone cuando alguien agrega la página a su pantalla de inicio. Este último va con fondo opaco, porque iOS pinta de negro las zonas transparentes.
 - **SVG de verdad:** los dos SVG tienen que ser vectoriales de verdad, sin texto guardado como texto (el navegador lo dibujaría con otra fuente) y sin una imagen PNG metida adentro (hay conversores que hacen eso, y el archivo termina pixelado igual). Cada uno tiene que pesar menos de 10 KB; si pesa más, suele ser metadata del editor, y SVGO la saca (`npx svgo archivo.svg`, sin instalar nada).
- **Dónde viven:**
 - `apps/web/src/assets/logo.svg`. La carpeta `assets/` todavía no existe: se crea dentro de `src/`, al lado de `components/`. El navbar lo importa (desde `src/components/`, `import logoUrl from '../assets/logo.svg'`) y lo muestra con `alt="Scaps"`. Va en `src/assets/` y no en `public/` porque así Vite le cambia el nombre en cada build, y el navegador nunca muestra una versión vieja guardada en caché.
 - `favicon.svg`, `favicon-32x32.png` y `apple-touch-icon.png`, en `apps/web/public/`. Estos sí van en `public/`, porque `index.html` los pide por una dirección fija. En `apps/web/index.html` se declaran así, reemplazando el ícono de Vite si todavía está, y con el título de la pestaña en "Scaps":

```
<link rel="icon" href="/favicon.svg" type="image/svg+xml" />
<link rel="icon" href="/favicon-32x32.png" type="image/png" sizes="32x32" />
<link rel="apple-touch-icon" href="/apple-touch-icon.png" />
<title>Scaps</title>
```
 - Los archivos originales, tal como salieron antes de adaptarlos, en `docs/marca/`, igual que los `.glb` originales viven en `docs/glb/`.
- **2 • Las rutas del MVP.** Con el recorte de alcance, el checkout y la libreta de direcciones pasaron a la segunda etapa, pero sus rutas siguen declaradas desde el Sprint 1: `/checkout` y `/mis-direcciones`, cada una con su página vacía. Sacarlas del router, borrar esas páginas y quitar cualquier enlace que apunte a ellas. El ERD y el contrato no se tocan: cuando vuelva el checkout, las rutas se declaran de nuevo en un minuto.
- **3 • El navbar responsive.**
 - **Contenido:** el logo, que lleva a la landing, y Catálogo. Sin sesión, Ingresar y Registrarse; con sesión, el nombre del usuario y Cerrar sesión. El Carrito se suma en el Sprint 3 y el acceso al panel de admin en el 4, cuando existan. "Mis pedidos" y "Mi perfil", que muestra la guía, no van: salieron del MVP.
 - **La sección actual se resalta**, para que el usuario sepa dónde está. React Router trae `NavLink`, que ya sabe cuál es el enlace activo.

- **En el celular:** el logo a la izquierda y, a la derecha, el botón de menú (las tres rayitas), que despliega los enlaces debajo de la barra. Se cierra al elegir un enlace y con Esc, y lleva `aria-expanded` y `aria-label` para los lectores de pantalla. No hace falta ninguna librería: es un estado de abierto o cerrado.
- **Hecho cuando:** el logo y los tres archivos del favicon están donde se indica arriba, con esos formatos y tamaños; ningún SVG tiene etiquetas `<text>` ni `<image>` (se ve abriéndolo con un editor de texto) y cada uno pesa menos de 10 KB; el logo se distingue sobre el fondo del navbar y se ve nítido con el zoom del navegador al 200 %; la pestaña muestra el favicon nuevo, que se distingue con el navegador en tema claro y en tema oscuro, y el título “Scaps”; `/checkout` y `/mis-direcciones` caen en el 404 y ningún enlace lleva a ellas; el navbar muestra los enlaces que corresponden con sesión y sin ella, y resalta la sección actual; y en los cuatro anchos de prueba se ve completo, sin scroll horizontal, con el menú desplegable en el celular, que se cierra al elegir un enlace y con Esc.

[setup] Footer con los créditos de los modelos 3D

setup

prioridad-alta

- **Qué hacer:** sumar al layout un footer que se vea en todas las pantallas, con la marca, el año, una línea que aclare que es un proyecto demo con productos ficticios (para que nadie tome la tienda por real) y un enlace **Créditos de los modelos 3D**.
- **La página de créditos:** el enlace lleva a una ruta nueva, `/creditos`, que muestra los cuatro textos de atribución tal cual están en `docs/glb/Creditos_Modelos_3D_Scaps.pdf` (el apartado “Texto para atribución” de cada modelo), con sus enlaces activos: el de la página del modelo y el de la licencia. Es texto fijo en el frontend: no hace falta tocar la base ni la API. Si algún día se suma un modelo, se actualizan los dos, el PDF y la página.
- **Siempre abajo:** si una página tiene poco contenido, como el 404 o el catálogo vacío, el footer igual queda al fondo de la pantalla y no flotando en el medio. Se resuelve armando el layout como una columna flex de alto mínimo `min-h-screen`, con el contenido en `flex-1`.
- **En el celular:** el footer se apila en una columna.
- **Hecho cuando:** el footer aparece en todas las pantallas y queda al fondo incluso en el 404; `/creditos` muestra los cuatro textos de atribución, y sus enlaces abren la página de cada modelo y la de la licencia; y en los cuatro anchos de prueba se ve completo, sin scroll horizontal.

Por qué no es opcional: la licencia CC BY permite usar un modelo solo si se le da crédito a su autor, y el plan exige esa atribución dentro de la app. En este sprint el visor publica los modelos en la landing y en la ficha, y las fotos del catálogo salen de esos mismos modelos: los créditos tienen que estar publicados cuando salga el visor. Los cuatro modelos del catálogo son CC BY.

Backlog — esperan que se libere una dependencia

[catalogo] Grilla del catálogo

catalogo

prioridad-alta

- **Depende de:** Piezas compartidas de producto.
- **Qué hacer:** construir la pantalla `/catalogo`: pedir la primera tanda a `GET /products` y mostrar una card por producto, con la imagen principal (`<ProductImage>` en un área cuadrada, 1:1, el mismo formato que las fotos), el nombre, el precio (`formatPrice`) y el botón **Ver detalles**. La card entera es un enlace a `/producto/:slug`. Si `stock` es 0, la card suma la etiqueta **Sin stock**, pero **Ver detalles** sigue habilitado.

- **Tandas de 24 productos** (`limit=24`): 24 se divide justo por 4, 3 y 2, así la última fila queda completa en escritorio (4 columnas), tablet (3) y celular (2). Dejarlo en una constante, `PAGE_SIZE` : la tarea de Cargar más la usa.
- **Responsive**: 2 columnas de base, 3 desde `md` (768 px) y 4 desde `lg` (1024 px), con `grid-cols-2 md:grid-cols-3 lg:grid-cols-4` . En el celular la separación entre cards baja de 24 a 12 px (`gap-3 md:gap-6`) y la card no muestra el botón Ver detalles (`hidden md:block`): la card entera ya es un enlace, y a ese ancho el botón ocupa alto y compite con el nombre y el precio. En tablet y escritorio el botón se queda, como en la guía.
- **Cards parejas**: todas las cards de una fila tienen que quedar del mismo alto, con el botón alineado abajo, aunque un nombre sea más largo que otro. El nombre ocupa como máximo dos renglones (Tailwind trae `line-clamp-2` , que corta con "..."), y la card es una columna flex con el botón empujado al fondo (`mt-auto`). Es lo que hace que la grilla se lea como un catálogo prolijo y no como cajas sueltas.
- **Encabezado y pie**: arriba, "Catálogo" con la cantidad total de productos, que sale de `meta.total` . Abajo, cuántos se están mostrando: "Mostrando 24 de 30 productos", o "Fin del catálogo · 4 de 4 productos" cuando ya están todos. Así, aunque todavía no exista el botón Cargar más, nunca quedan productos ocultos sin aviso.
- **Estados**: cargando (si tarda, el cliente HTTP ya muestra "despertando el servidor"), error con un botón **Reintentar**, y catálogo vacío ("Todavía no hay productos").
- **Hecho cuando**: los cuatro productos del seed se ven con imagen, nombre y precio con formato; poniéndole stock 0 a uno en tu base de desarrollo (desde `prisma studio`) aparece **Sin stock** y su card sigue llevando a la ficha; el pie dice "Fin del catálogo · 4 de 4 productos"; con el backend apagado aparece el error con **Reintentar**, no una pantalla rota; en los cuatro anchos de prueba la grilla muestra 2, 2, 3 y 4 columnas, sin scroll horizontal y sin el botón en el celular; y cambiándole a un producto el nombre por uno largo (en tu base de desarrollo), su card corta el nombre en dos renglones y la fila sigue pareja.

[catalogo] Ficha de producto: datos y galería

catalogo prioridad-alta

- **Depende de**: Piezas compartidas de producto.
- **Qué hacer**: construir la pantalla `/producto/:slug` con los datos de `GET /products/:slug` :
 - breadcrumb "Catálogo / Nombre del producto", con Catálogo como enlace;
 - nombre, precio (`formatPrice`), stock ("Stock: 14", o **Sin stock** si es 0) y descripción;
 - galería: la imagen grande (`<ProductImage>` en 700 × 520) y al costado las miniaturas de todas las imágenes, en el orden del campo `orden` . Arranca mostrando la principal (`es_principal: true`); tocar una miniatura la pasa a la imagen grande, y la activa queda marcada. Las miniaturas son botones, así también funcionan con el teclado;
 - el título de la pestaña del navegador con el nombre del producto (por ejemplo, "Trucker Negra | Scaps");
 - "Producto no encontrado", con un enlace al catálogo, cuando la API responde `404` : slug inexistente o producto dado de baja.
- **Dos bugs clásicos de una SPA** (una app de una sola página, como esta), que conviene evitar de entrada:
 - **La ficha tiene que abrir desde arriba**. El navegador no lo hace solo al cambiar de ruta: si venías scrolleado, la ficha aparece a mitad de página. Si el router es de los *data routers* (`createBrowserRouter`), alcanza con agregar `<ScrollRestoration />` ; si no, un `useEffect` que haga `window.scrollTo(0, 0)` al cambiar la ruta.

- **El estado no se arrastra de un producto a otro.** React Router reutiliza el mismo componente al pasar de una ficha a otra (por ejemplo, con Atrás y Adelante), y el estado queda del producto anterior: la galería podría apuntar a la cuarta imagen de un producto que tiene tres. Lo más simple es darle al componente de la página `key={slug}`, así React lo arma de cero con cada producto.
- **En el celular:** todo se apila en una columna. Arriba, la imagen grande a todo el ancho (con la proporción de 700 × 520); debajo, las miniaturas en una fila, con scroll horizontal si no entran; y después los datos.
- **Qué no va en esta tarea:** el botón Ver en 3D, que tiene su propia tarea, y el selector de cantidad con Agregar al carrito, que llegan con el carrito en el Sprint 3. Tampoco SKU, envío ni características (ver “Antes de arrancar”).
- **Hecho cuando:** los cuatro productos se abren por su slug con sus datos y su galería; las miniaturas cambian la imagen grande con el mouse y con el teclado; un slug inventado y un producto con `activo` en `false` en tu base de desarrollo muestran “Producto no encontrado”; entrar a una ficha desde una página scrolleada la muestra desde arriba; el título de la pestaña cambia con cada producto; y en los cuatro anchos de prueba la ficha se ve completa, sin scroll horizontal, con las miniaturas en fila en el celular.

[visor-3d] Visor del destacado en la landing

visor-3d prioridad-alta

- **Depende de:** Piezas compartidas de producto + Componente del visor 3D.
- **Qué hacer:** sumar a la landing del Sprint 1 el visor del producto destacado, que se pide a `GET /products/featured`. El texto del hero y el botón **Ver catálogo** se muestran desde el primer instante, sin esperar a la API; lo que cambia según la respuesta es el área visual:
 - **mientras llega la respuesta:** un indicador de carga;
 - **hay destacado:** el `<ModelViewer>` con su `glb_url`, sobre el fondo oscuro de la guía, con el nombre del producto como enlace a su ficha y el texto de ayuda “Arrastrá para rotar”;
 - **no hay destacado (404) o el pedido falla:** la imagen fija del hero del Sprint 1. Es el fallback que exige el contrato: la landing nunca puede verse vacía;
 - **hay destacado, pero su `.glb` no carga** (el visor dispara `onError`): en lugar del canvas, la imagen principal del destacado (la que tiene `es_principal: true`) con `<ProductImage>`. Ya viene en la respuesta, no hace falta otro pedido.
- **En el celular:** el hero se apila. Primero el texto y el botón Ver catálogo, que tienen que verse sin scrollear; debajo, el visor, con un alto limitado (ver la trampa del celular, más abajo).
- **Hecho cuando:** en local y en el preview de un PR se ve el destacado en 3D y se puede rotar; con el backend dormido, el hero aparece de entrada y el visor se suma cuando el backend despierta; editando datos en tu base de desarrollo desde `prisma studio` (y restaurándolos después), con `destacado` en `false` la landing muestra la imagen fija y con una `glb_url` rota muestra la imagen principal; en el celular se puede scrollear la landing entera sin quedar atrapado en el visor, y el botón Ver catálogo se ve sin scrollear; en los cuatro anchos de prueba no hay scroll horizontal; y la gorra se distingue bien del fondo oscuro.

La trampa del celular: `OrbitControls` se queda con los gestos táctiles que empiezan sobre el canvas, así que tocarlo rota el modelo en vez de scrollear la página. Si el visor ocupa toda la pantalla del celular, el usuario no tiene dónde apoyar el dedo para bajar: el visor nunca ocupa el alto completo, siempre tiene que quedar espacio para scrollear. **Y sobre el contraste:** si una gorra oscura se pierde contra el fondo, se ajustan las luces del componente del visor; una luz de contorno desde atrás suele alcanzar.

[catalogo] Búsqueda, filtros y orden

catalogo prioridad-alta

- **Depende de:** Grilla del catálogo.
- **Qué hacer:** sumar a la grilla los controles que ofrece el contrato: el buscador (parámetro `q`, se aplica al apretar Enter), el orden (`sort`, se aplica al elegirlo; opciones: Más recientes —la de por defecto—, Precio: menor a mayor, Precio: mayor a menor, Nombre: A-Z y Nombre: Z-A) y la barra de filtros (`precio_min`, `precio_max` y la casilla Con stock, que se aplican con el botón **Aplicar**).
- **El estado vive en la URL**, no en un `useState`: `/catalogo?q=trucker&sort=precio_asc`. React Router lo resuelve con `useSearchParams`. Así, al volver de una ficha con Atrás los filtros siguen puestos, y recargar la página no los borra.
- **Cuatro cuidados, porque la API es estricta:** ante un parámetro que no conoce, o con un valor inválido, responde `400`.
 - Al armar el pedido, mandar solo los parámetros que tienen valor: un `precio_min=` vacío ya es un `400`. `en_stock` va solo cuando la casilla está marcada.
 - Al leer la URL, descartar lo que no sea válido antes de llamar a la API. Cualquiera puede editarla a mano (`?precio_min=abc`), y eso no puede terminar en una pantalla de error.
 - Los campos de precio aceptan solo dígitos. Acá hay una trampa bien argentina: la gente escribe `10.000` para decir diez mil, y la API lo rechaza, porque lo lee como diez con tres decimales y el contrato admite hasta dos. Quitar los puntos resuelve el caso.
 - Si el mínimo es mayor que el máximo, avisarlo junto a los campos y no mandar el pedido.
- **Respuestas viejas:** si alguien cambia un filtro mientras otro pedido sigue en curso, la respuesta vieja no puede pisar a la nueva. Con una conexión lenta pasa: el primer pedido puede volver último. Se resuelve descartando la respuesta de un pedido que ya no es el actual, con un `AbortController` o con una bandera en la limpieza del `useEffect`.
- **Sin resultados:** “No encontramos productos con esos filtros”, con un botón **Limpiar filtros**.
- **En el celular:** el buscador ocupa todo el ancho, y debajo van el orden y un botón **Filtros** que abre y cierra la barra de filtros en el mismo lugar, sin ventana emergente. Plegada, deja ver los productos en la primera pantalla; si hay filtros aplicados, el botón lo indica (“Filtros · 2”), para que nadie se pregunte por qué ve menos productos. Los campos de precio llevan `inputMode="numeric"`, que en el celular abre el teclado numérico. Los controles mantienen la altura de la guía (44 px los campos, 48 px los botones), que además es un tamaño cómodo para el dedo.
- **Hecho cuando:** cada control funciona solo y combinado con los demás, y la URL lo refleja; volver de una ficha con Atrás y recargar la página conservan los filtros; escribir `10.000` en un precio funciona; un mínimo mayor que el máximo muestra el aviso sin llamar a la API; una URL editada a mano con valores inventados no muestra una pantalla de error; y con la red en *Slow 3G* (DevTools → Network) y cambios de filtro rápidos, lo que queda en pantalla corresponde al último filtro elegido; y en el celular la barra se pliega y despliega con el botón Filtros, el precio abre el teclado numérico y no hay scroll horizontal en ninguno de los cuatro anchos.

[visor-3d] Ver en 3D en la ficha

visor-3d prioridad-alta

- **Depende de:** Componente del visor 3D + Ficha de producto.
- **Qué hacer:** sumar sobre la imagen grande de la ficha un botón **Ver en 3D**. Al tocarlo, en el mismo lugar y del mismo tamaño que la imagen, aparece el `<ModelViewer>` con el `glb_url` del producto,

sobre fondo blanco: las fotos son renders de esos mismos modelos sobre blanco, así que el paso de foto a 3D no salta. Un botón **Ver fotos**, o tocar cualquier miniatura, vuelve a las imágenes.

- El `.glb` se descarga recién al tocar Ver en 3D, no al abrir la ficha: algunos modelos pesan varios megas, y la mayoría de las visitas no van a abrir el 3D.
- Si el modelo no carga (`onError`), la ficha vuelve a las fotos con un aviso breve: “No pudimos cargar el modelo 3D”.
- Al pasar a otro producto, la ficha arranca en las fotos. Con el `key={slug}` de la tarea de la ficha, esto sale solo.
- **Hecho cuando:** al abrir una ficha, la pestaña *Network* no muestra la descarga del `.glb` hasta tocar Ver en 3D; se puede ir y volver entre fotos y 3D; cada producto muestra su propio modelo; con una `glb_url` rota en tu base de desarrollo aparece el aviso y vuelven las fotos; en el celular el visor ocupa el lugar de la imagen grande, a todo el ancho, y se puede scrollear la ficha sin quedar atrapado en él (la misma trampa que en la landing); y todo funciona en el preview de un PR.

[catalogo] Cargar más

catalogo

prioridad-media

- **Depende de:** Búsqueda, filtros y orden.
- **Qué hacer:** sumar al pie de la grilla un botón **Cargar más** que pide la página siguiente (`page + 1`) y agrega esos productos debajo de los que ya están. Es la forma de “vista única con scroll continuo” que eligió el equipo: la lista crece sin números de página, pero la carga la dispara el usuario, en vez de un detector de “llegué al final” que carga solo. Ese detector es donde aparecen los bugs difíciles: cargas duplicadas, o una lista que se traba cuando una tanda no llena la pantalla.
- **Reglas:**
 - el botón se muestra mientras `meta.page` sea menor que `meta.total_pages`, y se deshabilita mientras hay un pedido en curso, para no pedir dos veces la misma página;
 - si cambia la búsqueda, un filtro o el orden, la lista vuelve a la primera página y **reemplaza** lo cargado, en vez de sumarse;
 - el pie (“Mostrando 48 de 60 productos”) se actualiza con cada tanda;
 - en el celular, el botón ocupa todo el ancho de la grilla.
- **Lo que no se guarda:** las páginas cargadas no van a la URL, así que al volver de una ficha la lista arranca de nuevo desde la primera tanda. Es una concesión aceptada: con este tamaño de catálogo no se nota.
- **Hecho cuando:** con `PAGE_SIZE` bajada a 2 (los cuatro productos del seed no alcanzan para probar con 24), se muestran 2, **Cargar más** trae los otros 2 y el botón desaparece; cambiar un filtro con todo cargado vuelve a la primera tanda, sin productos duplicados; y `PAGE_SIZE` vuelve a 24 antes de abrir el Pull Request.

Por qué prioridad media: con los productos de demo el botón nunca aparece, así que si el sprint se aprieta es la primera tarea que se mueve, sin romper nada. Mientras tanto, el pie de la grilla avisa si hay más productos de los que se muestran.